

InvestSim

Personalized Full Stack FinTech System — MERN

Field	Value
Roll Number	23i-5544
Student Name	
Section	
Course	Web Programming — BS FinTech Sem 6
Instructor	Arsalan Khan FAST NUCES
Domain (digit■=4)	Investment Simulator
Logic (digit■=4)	Alert System
Complexity ((4+4)%3=2)	Advanced — 3+ Collections

1. Problem Definition

Modern retail investors face two critical challenges: **tracking the future growth potential of their investments** and **receiving timely warnings when portfolio risk thresholds are breached**. Without automated monitoring, investors either miss important warning signals (such as a 15%+ loss on a single position) or fail to realise when a target return has been reached.

InvestSim solves both problems by providing: (1) an Investment Simulator that uses the Compound Interest Formula to project portfolio growth across three return scenarios (conservative 4%, moderate 8%, aggressive 14%), and (2) an intelligent Alert Engine that automatically monitors every portfolio update and triggers threshold-based alerts (stop-loss, price-drop, price-surge, target-reached, high-risk, low-diversification) — displayed in real time on the React dashboard.

2. System Architecture

MERN Stack Flow

The application follows a standard 3-tier MERN architecture:

Layer	Technology	Responsibility
Frontend	React 18 + React Router 6	UI, protected routes, charts (Recharts)
Backend	Node.js + Express 4	REST API, business logic, middleware
Database	MongoDB Atlas + Mongoose	4 collections with references
Auth	JWT (jsonwebtoken)	Stateless session across refreshes
Deployment	Vercel / Render / Atlas	Frontend / Backend / Database

Request Flow: React (axios) → Express routes → Auth middleware → Validation middleware → Controller/Logic → Mongoose → MongoDB Atlas → JSON response → React state update → UI re-render.

3. Database Design

3.1 Collections Overview

Collection	Purpose	Relationships
User	Stores credentials, risk profile	Root — referenced by all others
Investment	Individual asset holdings + simulation data	References User
Portfolio	Aggregated stats per user	References User (1-to-1)

Alert	Auto-generated threshold alerts	References User + Investment
-------	---------------------------------	------------------------------

3.2 Key Schema Fields

Collection	Key Fields
User	name, email, password (bcrypt), riskProfile, timestamps
Investment	user (ref), assetName, assetType, quantity, purchasePrice, currentPrice, amountInvested, currentValue, ...
Portfolio	user (ref, unique), totalInvested, totalCurrentValue, riskScore, diversificationScore, assetAllocation (by type)
Alert	user (ref), investment (ref, nullable), alertType, severity, message, triggerValue, actualValue, isRead

3.3 Referencing vs Embedding — Justification

Choice: Referencing (ObjectId references) across all collections.

Reason 1 — Independent querying: Investments are filtered, sorted, and aggregated independently of the User document. Embedding investments inside User would require pulling the entire User object every time.

Reason 2 — Document size growth: A user can have many investments and many alerts over time. Embedding would cause the User document to grow unboundedly, violating MongoDB's 16 MB document size limit.

Reason 3 — Dual references in Alert: An Alert references both a User and an Investment. It is impossible to embed such a document in two parents simultaneously — referencing is the only viable design.

Reason 4 — Update efficiency: When a stock price updates, only the Investment document changes. With embedding, the entire User document would need to be rewritten on every price update.

4. Core FinTech Logic Explanation

4.1 Investment Simulation Engine

The simulation uses the **Compound Interest Formula**: $FV = PV \times (1 + r)^t$, where PV is the current portfolio value, r is the annual return rate as a decimal, and t is the number of years.

Three scenarios are pre-defined: Conservative (r=0.04), Moderate (r=0.08), Aggressive (r=0.14). For each investment, the backend computes projected values at t=1, t=3, and t=5. These are stored on the Investment document and visualised on the Simulate page as a multi-line Recharts chart.

Example: An investment currently worth \$10,000 at moderate rate (8%):

1 Year: $\$10,000 \times (1.08)^1 = \$10,800$

3 Years: $\$10,000 \times (1.08)^3 = \$12,597$

5 Years: $\$10,000 \times (1.08)^5 = \$14,693$

4.2 Alert Engine (Threshold-based Decisions)

The Alert Engine runs automatically on every portfolio change (add, update, sell). It evaluates each investment and portfolio-level metrics against hardcoded thresholds, then bulk-inserts Alert documents into MongoDB.

Alert Type	Threshold Condition	Severity
stop_loss	Return % < -15%	Critical ■
price_drop	Return % < -5%	Warning ■
price_surge	Return % > +20%	Info ■
target_reached	Return % > +50%	Warning ■
high_risk	Portfolio risk score > 70	Warning ■
low_diversification	Diversification < 40	Info ■

4.3 Risk Scoring

Each asset type is assigned a risk weight: Crypto=90, Stock=60, ETF=40, Mutual Fund=30, Bond=15. The portfolio risk score is computed as a weighted average where each investment's weight is its proportion of total portfolio value. This produces a score from 0–100 that reflects true exposure.

4.4 Diversification Scoring

Diversification score = (distinct asset types / 5) × 100. If a user holds only stocks, the score is 20/100. If they hold stocks, bonds, and crypto, the score is 60/100. A score below 40 triggers an alert.

5. Query Explanation

Query 1 — Aggregation: Performance by Asset Type

Route: GET /api/investments/stats/aggregated

This MongoDB aggregation pipeline groups all active investments by assetType, then computes: total amount invested, total current value, count of holdings, and average expected return rate per group. An addFields stage computes profitLoss and returnPercentage. Results are sorted descending by return and displayed as a bar chart on the Dashboard.

Sample result: [{_id: 'crypto', totalInvested: 5000, totalCurrentValue: 6200, returnPercentage: 24.0}, {_id: 'stock', totalInvested: 8000, totalCurrentValue: 8400, returnPercentage: 5.0}]

Query 2 — Ranking: Portfolio Leaderboard

Route: GET /api/portfolio/rankings

This aggregation pipeline: (1) matches portfolios with totalInvested > 0, (2) computes returnPercentage per portfolio, (3) sorts descending by return, (4) performs a \$lookup join with the users collection to fetch

investor names, (5) projects out sensitive fields (password, email). The current user's rank is computed in application code by finding their index in the sorted array.

6. Security Analysis

Security Measure	Implementation	Layer
JWT Authentication	jsonwebtoken, 7-day expiry, Bearer header	Backend middleware
Password Hashing	bcryptjs, 12 salt rounds	User model pre-save hook
Input Validation	express-validator on all POST/PUT routes	Backend middleware
Protected Routes (FE)	React component checks useAuth() before render	Frontend
Protected Routes (BE)	protect middleware on all /investments, /portfolio, /alerts	Backend
User data isolation	Every query filters by req.user._id	All controllers

Known Vulnerability & Fix

Vulnerability: The current implementation does not rate-limit the login endpoint, making it susceptible to brute-force password attacks. An attacker could attempt thousands of password combinations against a known email address.

Fix: Implement express-rate-limit middleware on the /api/auth/login route, allowing a maximum of 10 attempts per IP per 15 minutes. Additionally, implement account lockout after 5 consecutive failures using a loginAttempts counter on the User model.

7. Scalability Discussion

What would break at 10,000 users?

Bottleneck	Problem at Scale	Fix
Portfolio rankings query	Full collection scan on every request	Add compound index on totalInvested + user; cache rankings
Alert bulk inserts	insertMany on every price update is expensive	Use message queue (Bull/BullMQ + Redis) to process alerts
No pagination	GET /investments returns all records	Add limit/skip/cursor pagination to all list endpoints
MongoDB Atlas free tier	512 MB storage limit	Upgrade to M10 cluster; archive old alerts to cold storage
Single server	No horizontal scaling	Containerise with Docker; deploy on Railway/Render with auto
JWT no invalidation	Cannot log out all sessions on breach	Store JWT JTI in Redis blacklist; or use short-lived tokens + re